AI Engineering and Claude Architect · Part II, Retrieval

RAG, Lesson 6

Evaluating RAG

The dashboard said 67% correct. Policy lookups were at 88%, error codes at 50%, and multi-document questions at 50%, for two completely different reasons. The single number hid all of it, and pointed at no fix.

Module	AI05 of 13
Lesson	6 of 7
Prerequisites	AI01 L4, AI04 L3, AI05 L4, HLD M21B (recall vs faithfulness)
Exam relevance	RAG metrics are out of scope. **TS 5.5's stratified measurement applies directly**, and so does independent review (TS 4.6)
Reading time	~40 minutes

Contents

1. Evaluating RAG

Lesson 6 of 7 · RAG

Evaluating RAG

Key idea

RAG has two halves that fail independently, so score retrieval and generation separately, and report by question type. An overall number averages unlike things and tells you nothing to do next. **Test whether the system says "I don't know" when it should, and calibrate any LLM judge against humans.**

> *Before: HLD M21B's 2×2 of recall and faithfulness, and AI04 L3's recall@k and MRR. Teaches: building a golden set, claim-level faithfulness, abstention, sample sizes, judge calibration and online signals. Used by: AI05 L7, AI10 L2–L3, AI13 L4.*

The hook: one number, no direction

Here's a harness on 100 **scripted** cases. These results are invented to test the harness, and they say nothing about any model or retriever. Real output from `code/ai05-rag/.../RagEval.java`:

Segment	cases	recall	faithful	correct	diagnosis
ALL	100	0.63	0.69	0.67	fix ingestion or retrieval first
policy lookup	40	0.93	0.90	0.88	healthy
error code	20	0.50	0.50	0.50	fix ingestion or retrieval first
multi-document	20	0.80	0.55	0.50	fix generation
no answer exists	20	0.00	0.60	0.60	abstain rate is the metric

Key idea

The overall row points at retrieval, yet multi-document questions need the opposite fix: retrieval found the material, and generation failed to use it. The two segments at 50% correct have different causes. And the no-answer segment's recall of 0 is meaningless, because no right chunk exists. It drags the overall recall down: without it, recall is 63 of 80, or 0.79.

Intuition: a hospital's quality report

A hospital doesn't report one survival number. A hospital treating harder cases would look worse, and a failing cardiac unit could hide behind a strong maternity ward. It reports by department. It also separates diagnosis from treatment, since each needs a different remedy. RAG's segments are its departments, and retrieval and generation are its diagnosis and treatment.

Mechanism: what to measure

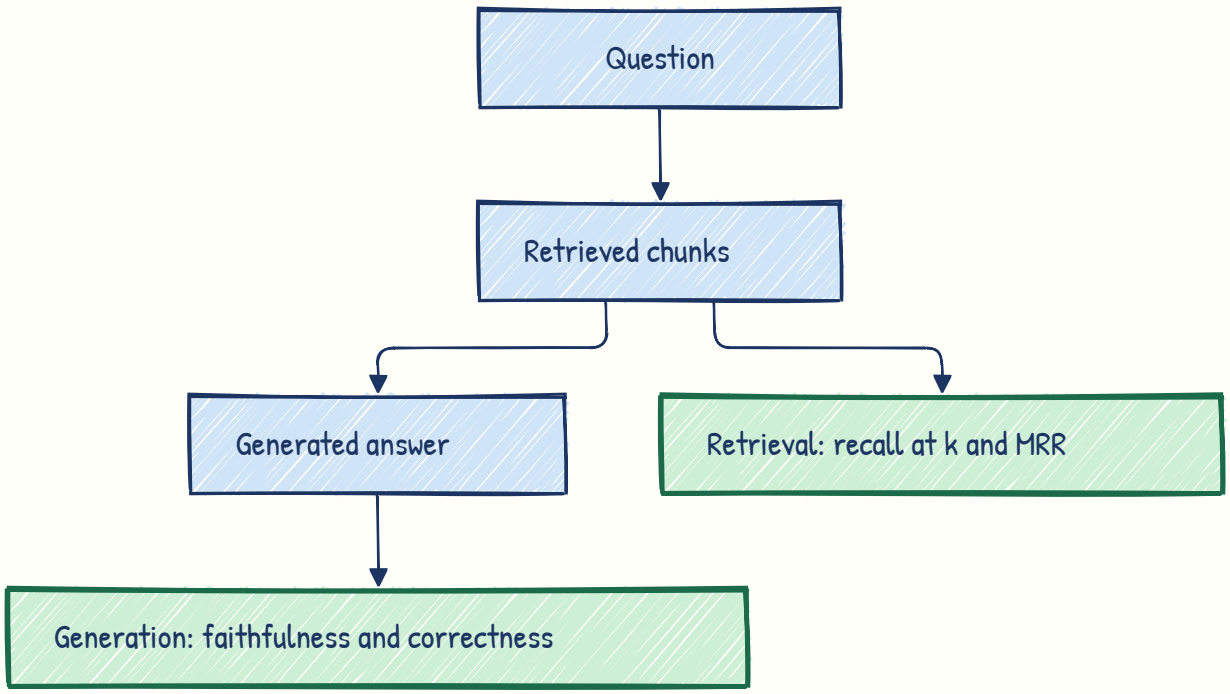


Figure 1. Where each metric attaches. Retrieval is measured against known relevant chunks, and generation against the retrieved context and the correct answer.

Metric	Question	Needs
Recall@k	Was the right chunk in the top k?	Labeled chunks
MRR (AI04 L3)	How high was the first right chunk?	Labeled chunks
Context precision	How many retrieved chunks were relevant?	Labels or a judge
Faithfulness	Is each claim supported by the retrieved text?	Context, plus a judge
Answer relevance	Does it answer what was asked?	A judge
Correctness	Does it match the known answer?	A reference answer
Abstention	Does it refuse when no answer exists?	Unanswerable questions
Citation support	Does the cited passage back the claim?	Sampling, with a judge

Ragas (Es and colleagues, 2023) proposed a reference-free framework covering retrieval, faithfulness and answer quality. That's useful when you lack labeled answers. But a reference-free faithfulness score can't tell you whether a faithful answer is correct, since a wrong source supports a wrong answer perfectly.

Why correctness and faithfulness differ

Faithful?	Correct?	Likely meaning	Fix
Yes	Yes	Working	Nothing
Yes	**No**	Wrong or stale source	Retrieval, or the data
No	Yes	Model knowledge, by luck	Generation. **Don't count this as a win**
No	No	Both	Start with retrieval

The third row is easy to miss. The answer's right, but it didn't come from your documents, so you can't rely on it holding.

Measure faithfulness per claim

A whole answer can't be scored fairly as one pass or fail. Break it into claims, then check each one.

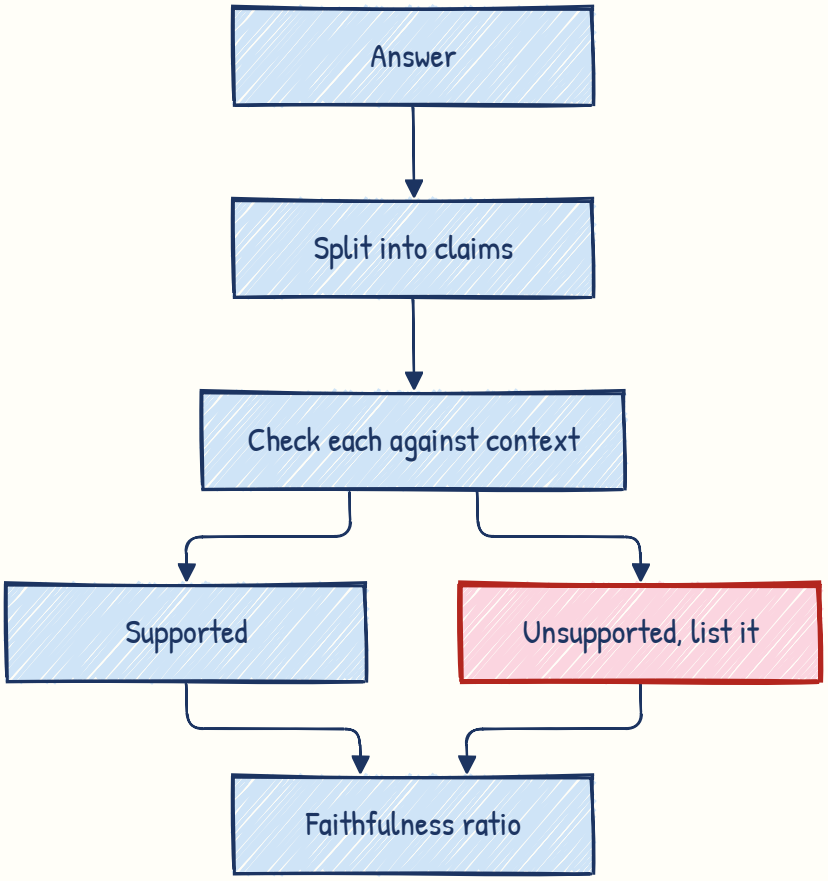


Figure 2. Claim-level faithfulness. The answer is split into separate claims, each is checked against the retrieved context, and the result is a ratio with named failures.

This gives you something actionable: not a score of 0.7, but **the exact sentence that isn't supported.**

Building a golden set

Source	Good for	Risk
Real logged questions	Actual vocabulary and distribution	Privacy, so redact
Support tickets	Hard, real cases	Answers may be stale
Subject experts	Critical questions	Limited, and expensive
Generated from your documents	Fast coverage	**Too easy**, since they echo the source wording
Past failures	Regression protection	None. Always add them

Label chunks by stable IDs, and expect churn. When documents change, chunk IDs change. Label by document and section, or keep expected text, so that labels survive a re-chunk.

Required segments

Segment	Why
By question type	Different failures
No answer exists	Tests abstention, which many sets omit
Exact identifiers	Tests keyword retrieval
Multi-document	Tests synthesis
Follow-ups	Tests rewriting
By permission group	Tests access. **Include a case where the answer exists but this user mustn't see it**
Superseded policy	Tests freshness

How many cases?

For a rate `p` on `n` cases, an approximate 95% margin of error is `1.96 × √(p(1-p)/n)`.

Cases	True rate	Margin of error
20	50%	±22%
40	90%	±9%
100	90%	±6%
385	50%	±5%

Small segments are noisy

With 20 cases, 50% could plausibly be anywhere from about 28% to 72%. So don't celebrate a two-point change. Use the same cases before and after, and a small set is still good for finding failures, even if it's poor at measuring small differences. Grow the segments that matter most.

LLM judges need calibrating

Judging thousands of answers by hand doesn't scale, so teams use a model as a judge. But a judge is itself a classifier, with its own error rate.

An illustrative calibration. Humans label 200 claims, finding 30 unsupported. The judge flags 34: 24 of those 30, plus 10 that humans called supported.

Measure	Calculation	Value
Judge recall	24 of 30	**0.80**
Judge precision	24 of 34	**0.71**
Unsupported rate, humans	30 of 200	15%
Unsupported rate, judge	34 of 200	17%

Note the trap. The two rates look close, 15% and 17%, yet the judge missed 6 real failures and invented 10 false ones. Totals can look well calibrated while individual judgments are poor.

Practice	Why
Compare with human labels regularly	The judge can drift, and models change
Use a separate context from the generator	Self-review is weaker (TS 4.6)
Require evidence: "quote the supporting text"	Verifiable, and it forces reasoning
Give a clear rubric, with examples	TS 4.1, explicit criteria
Sample what passed, too	Measures what the judge misses

The same logic applies to confidence in extraction: **TS 5.5** says to calibrate confidence on labeled sets, and to sample high-confidence items, using stratified sampling.

Online signals

Offline sets can't see tomorrow's questions.

Signal	Tells you	Caution
Thumbs and comments	Direct dissatisfaction	Few people use them
Reformulated question	The answer probably failed	Might be a new question
Citation clicks	Trust or doubt	Ambiguous
"I don't know" rate	Coverage gaps, or over-caution	Watch it both ways
Escalation rate	Failures reaching humans	Depends on the domain
Sampled review	The real quality number	**Costs human time**

LinkedIn's experience shows the cost. Human linguists evaluated up to 500 conversations a day, annotators gave results in about a day, and some metrics took three days or more. So they used a tiered approach to balance speed and depth.

Close the loop: a failure found in production becomes a test case. Over time, the set comes to reflect reality.

How Anthropic and the field approach it

Need	Approach	Source
Compare retrieval methods	Top-20 failure rate, as in the published 5.7% to 1.9%	Anthropic's Contextual Retrieval
Reference-free metrics	Ragas	Es and colleagues
Human evaluation at scale	Linguists plus tiered metrics	LinkedIn
Segment measurement	Stratified sampling, field-level calibration	CCAR-F TS 5.5

Anthropic's result is a good model of honest reporting: a defined metric, on stated datasets, with its baseline shown. But its figures are from those datasets, so **measure on yours.**

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: eyeball a few questions. It's quick and unrepeatable.

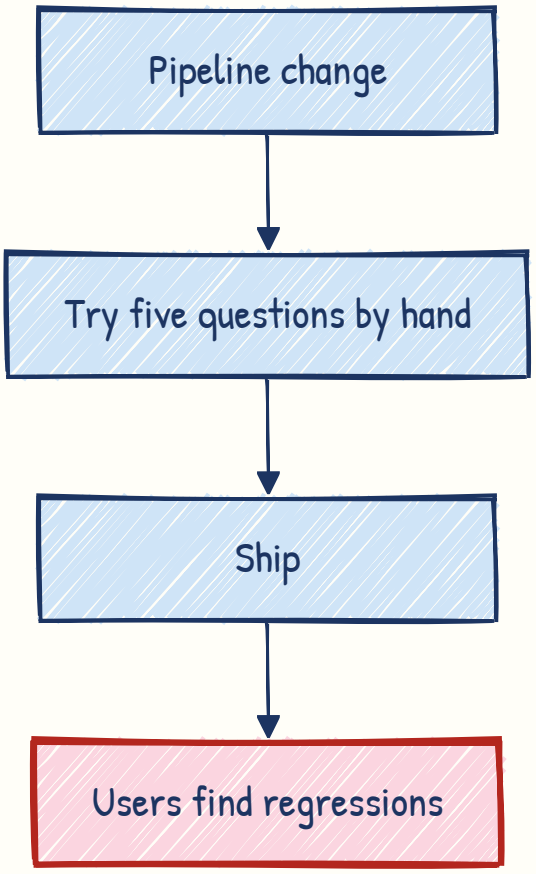


Figure 3. Baseline. A person tries some questions, so regressions go unnoticed until users find them.

Improved: an automated golden set in CI. Regressions are caught. New problem: **the set goes stale, and judges drift unnoticed.**

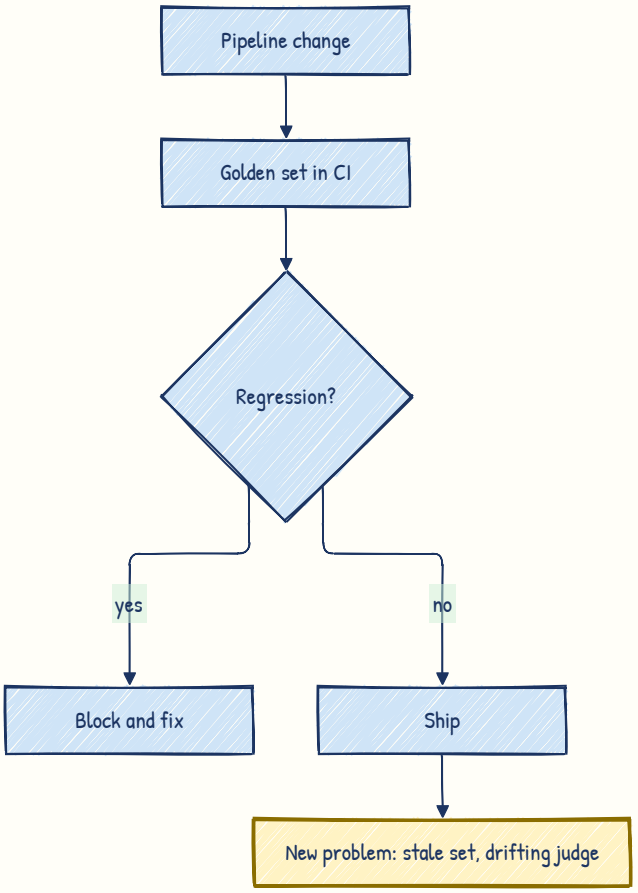


Figure 4. Improved. Each change runs against a fixed set, which slowly stops resembling real questions.

Production: a loop.

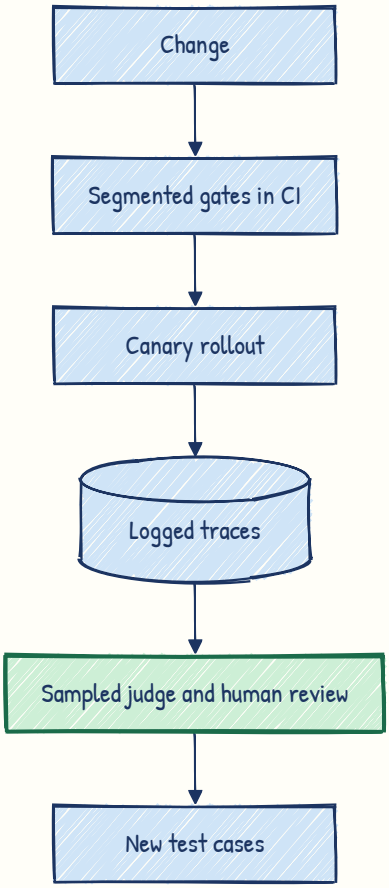


Figure 5. Production. Changes pass offline gates, go out gradually, and production samples are reviewed, with failures returning to the golden set.

Component	Why it exists	How it communicates
Segmented gates	Each segment has its own threshold	CI job, blocking
Trace log	Lets you replay and diagnose	Async, with question, rewrite, chunk IDs, answer, versions
Judge	Scale	Batch API (50% cheaper), since it's not urgent
Human review	Ground truth, judge calibration	Review queue, stratified sampling
Canary	Finds what offline sets can't	Traffic split

The cost is often small next to an incident. Illustratively, 300 cases at 4,000 input and 400 output tokens on Sonnet 5.5 cost $3.60 per run, and three judge calls per case on Haiku 5.5 add $0.18. That's before any Batch API discount.

Failures

Failure	Containment
Set no longer resembles traffic	Refresh it regularly from production samples
Judge drifts after a model change	Pin versions, and re-calibrate
Overall score hides a segment	Gate per segment
Team tunes to the test set	Keep a held-out set
Labels break after a re-chunk	Label by document and section
Logs hold sensitive content	Redact, and limit retention (AI09 L4)

Trade-offs

Decision	Gain	Cost
Human labels	Ground truth	Slow and expensive
LLM judge	Scale	Its own errors, so it needs calibrating
Reference-free metrics	No labels needed	Can't detect a faithful wrong answer
Large golden set	Precise measurement	Cost per run, and upkeep
Per-segment gates	Catch hidden regressions	Noisy, if segments are small

When you can go lighter: an internal tool with low stakes needs fewer cases, but it still needs the two-halves split, and no-answer cases.

Common mistakes

1. **Reporting a single number.** It hides segments.
2. **Mixing no-answer questions into recall.** They have no right chunk.
3. **Only using generated questions.** They echo the document's wording.
4. **Counting a correct but unfaithful answer as a win.** It may not hold.
5. **Trusting a judge unchecked.** Calibrate it on human labels.
6. **Comparing totals to check a judge.** Compare item by item.
7. **Reading two points on 20 cases as real.** The margin is ±22%.
8. **Never adding production failures.** The set goes stale.

What to say in an interview

"I'd score retrieval and generation separately, by question type, including questions with no answer and cases where the user lacks access. A faithful answer can still be wrong, so I'd keep reference answers for critical questions. I'd calibrate any LLM judge against humans item by item, since matching totals can hide many errors, and I'd feed production failures back into the set."

Certification lens

How this lesson maps to CCAR-F

TS 5.5 is the exam connection: aggregate accuracy can mask poor performance on particular types, so use stratified sampling and calibrated confidence. **TS 4.6** says a separate instance reviews better than the author, and **TS 4.1** calls for explicit criteria. RAG metric names themselves aren't tested.

These are original practice questions. Q4 is in the exam's style, and none are real or leaked.

Q Q1 · Troubleshooting · Medium. A segment has recall of 0.92 and faithfulness of 0.55. Another has recall of 0.48 and faithfulness of 0.91. What's wrong in each, and what do you do next?

A The first found the material, but the answers don't stick to it, so fix generation: instructions, context order, or the number of chunks. The second answers faithfully from whatever it got, but it's getting the wrong text, so fix ingestion and retrieval.

Q Q2 · Calculation · Easy. A question has 10 relevant chunks. Retrieval returns 8 chunks, with 3 relevant. What's context precision? If a larger k finds 7 of the 10 relevant ones, what's recall then?

A Context precision is 3 of 8, or **0.375**. Recall at the larger k is 7 of 10, or **0.70**.

Q Q3 · Best decision · Hard. Your faithfulness judge reports 90% this month, and 94% after a prompt change. There are 100 cases. Your manager wants to ship and announce the improvement. What do you say?

A At 100 cases, the margin is around ±6%, so independent samples wouldn't establish this. Using the same cases helps, so look at which ones changed. Check that the judge is unchanged, check segments for a hidden regression, and calibrate the judge against human labels. Ship only if no critical segment fell, and describe the gain as tentative.

Q Q4 · Exam style, scenario: Structured Data Extraction · Medium. An extraction pipeline shows 97% overall accuracy. The team proposes removing human review for all high-confidence results. What should you do first?

A. Remove review for documents with confidence above 0.9.

B. Validate accuracy by document type and field before reducing review, and keep stratified sampling of high-confidence results.

C. Raise the confidence threshold to 0.99 for all documents.

D. Keep reviewing everything indefinitely.

A **B** (TS 5.5). A and C trust a figure that may hide weak segments and an uncalibrated confidence score. D ignores what could safely be automated once it's shown.

Revision - Lesson 6

→ Score retrieval and generation separately, by question type.

→ Overall numbers can point towards the wrong fix.

→ Faithful isn't correct: a wrong source supports a wrong answer.

→ Correct but unfaithful isn't a win either.

→ Score faithfulness claim by claim, and list what's unsupported.

→ Golden sets need no-answer, exact-term, multi-document, follow-up, permission and stale-policy cases.

→ Generated questions are too easy. Add real ones and past failures.

→ Margin of error: ±22% at 20 cases and 50%, ±6% at 100 cases and 90%.

→ A judge is a classifier. Calibrate it item by item against humans.

→ Matching totals can hide many individual errors.

→ Run judges in a separate context, and use the Batch API.

→ Production failures become test cases.

Your notes